

HCP Vault Kubernetes Authentication Setup and Troubleshooting Guide

Author: Chirag Jain

Published Date: June 2026

Disclaimer

This document is based on testing and validation performed in a personal lab environment. Configuration details may vary depending on Kubernetes distribution, Vault version, network architecture, and organizational security requirements.

Environment:

- HCP Vault Dedicated
- K3s Kubernetes Cluster
- AWS EC2 (Ubuntu)
- Kubernetes Authentication Method

Overview

This document provides a step-by-step guide for configuring Kubernetes Authentication between HCP Vault and a K3s Kubernetes cluster. It also includes validation steps, troubleshooting techniques, and lessons learned during a hands-on lab implementation.

The objective of this guide is to:

- Configure Vault Kubernetes Authentication.
- Validate Service Account based authentication.
- Troubleshoot common authentication failures.
- Verify Kubernetes API connectivity and certificate validation.
- Provide a repeatable setup procedure for learning and testing purposes.

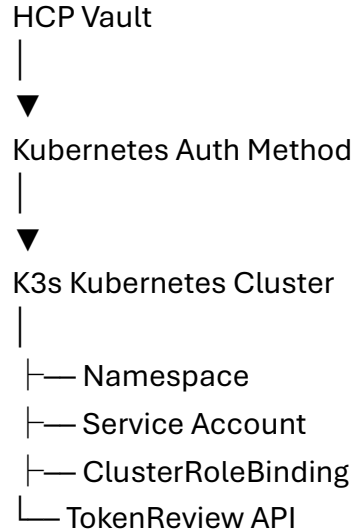
Target Audience

This guide is intended for:

- Vault Administrators
- Kubernetes Administrators
- DevOps Engineers

- Platform Engineers
- Site Reliability Engineers (SREs)
- Security Engineers

Architecture



Prerequisites

- Kubernetes cluster is up and accessible.
- Service Account exists or can be created.
- HCP Vault cluster is accessible.
- Vault CLI is installed.
- kubectl is installed and configured.
- Network connectivity exists between Vault and the Kubernetes API server.
- Access to create Service Accounts, ClusterRoleBindings, and Vault roles.

Why This Setup Is Required

Vault Kubernetes Authentication allows workloads running in Kubernetes to authenticate securely using Kubernetes Service Account tokens.

During authentication, Vault performs a TokenReview request against the Kubernetes API server to verify the presented JWT. Successful authentication depends on:

- Correct Service Account configuration.
- Proper RBAC permissions.
- Kubernetes API accessibility.
- Valid TLS certificates.
- Correct Vault role configuration.

The following sections describe the complete implementation and validation process.

Step 1: Create Namespace and Service Account

Purpose

Vault Kubernetes Authentication uses Kubernetes Service Account tokens to authenticate workloads running inside a Kubernetes cluster. A dedicated Service Account is required so that JWT tokens can be generated and validated by Vault during the authentication process.

Create Namespace

Create a namespace to host the Service Account used for Vault authentication testing.

```
kubectl create namespace <namespace-name>
```

Create Service Account

Create a Service Account that will be associated with Vault Kubernetes Authentication.

```
kubectl create serviceaccount <service-account-name> -n <namespace-name>
```

Verification

Verify that the Service Account has been created successfully.

```
kubectl get sa <service-account-name> -n <namespace-name>
```

Expected Output

```
NAME      SECRETS  AGE
<service-account-name>  0        10s
```

Explanation

The Service Account will be used to generate Kubernetes JWT tokens. During authentication, Vault validates these JWTs by communicating with the Kubernetes API server through the TokenReview API.

Without a valid Service Account, Vault Kubernetes Authentication cannot be configured or tested successfully.

Step 2: Grant TokenReview Permissions

Purpose

Vault validates Kubernetes Service Account tokens by calling the Kubernetes TokenReview API.

To perform TokenReview requests, the Service Account must be granted the system:auth-delegator ClusterRole.

Without this permission, Vault will not be able to validate JWT tokens and Kubernetes Authentication will fail.

Create ClusterRoleBinding

Create a file named binding.yaml with the following contents:

```
apiVersion: rbac.authorization.k8s.io/v1
```

```
kind: ClusterRoleBinding
```

```
metadata:
```

```
  name: <service-account-name>-sa-binding
```

```
roleRef:
```

```
  apiGroup: rbac.authorization.k8s.io
```

```
  kind: ClusterRole
```

```
  name: system:auth-delegator
```

```
subjects:
```

```
- kind: ServiceAccount
```

```
  name: <service-account-name>
```

```
  namespace: <namespace-name>
```

Apply Configuration

Apply the ClusterRoleBinding:

```
kubectl apply -f binding.yaml
```

Verification

Verify that the Service Account can perform TokenReview operations:

```
kubectl auth can-i create tokenreviews.authentication.k8s.io \
```

```
--as=system:serviceaccount:<namespace-name>:<service-account-name>
```

Expected Output

yes

Additional Validation

Verify that the ClusterRoleBinding exists:

```
kubectl get clusterrolebinding <service-account-name>-sa-binding
```

Expected output:

NAME	ROLE	AGE
<service-account-name>-sa-binding	ClusterRole/system:auth-delegator	10s

Explanation

The system:auth-delegator ClusterRole provides access to the Kubernetes TokenReview API.

During authentication, Vault sends the presented JWT token to the Kubernetes API server using a TokenReview request. Kubernetes validates the token and returns the authentication result to Vault.

If this permission is missing, Vault may fail authentication requests and return authorization or permission-related errors.

Step 3: Generate Reviewer JWT

Purpose

Vault requires a Kubernetes Service Account token to communicate with the Kubernetes API server and perform TokenReview operations.

This token is configured as the `token_reviewer_jwt` in the Vault Kubernetes Authentication configuration.

Generate Service Account Token

Generate a JWT token for the Service Account:

```
kubectl create token <service-account-name> -n <namespace-name>
```

Example Output

```
eyJhbGciOiJSUzI1NiIsImtpZCI6...
```

Copy and save the generated token securely. This token will be used later when configuring Vault Kubernetes Authentication.

Verification

Decode the JWT payload to verify the Service Account information:

```
TOKEN=$(kubectl create token <service-account-name> -n <namespace-name>)
```

```
echo $TOKEN | cut -d ' ' -f2 | base64 -d
```

Review the decoded output and verify that it contains:

- Service Account name
- Namespace
- Subject (sub)
- Issuer (iss)
- Audience (aud), if configured

Explanation

The generated JWT represents the Kubernetes Service Account identity.

When Vault receives a login request, it uses the configured `token_reviewer_jwt` to call the Kubernetes TokenReview API and verify the authenticity of the presented Service Account token.

Without a valid reviewer token, Vault cannot validate Kubernetes JWTs and authentication requests will fail.

Step 4: Retrieve Kubernetes CA Certificate

Purpose

Vault must trust the Kubernetes API server before it can communicate with it securely.

The Kubernetes CA (Certificate Authority) certificate is used by Vault to validate the TLS certificate presented by the Kubernetes API server during authentication and TokenReview requests.

Retrieve the Kubernetes CA Certificate

Run the following command to extract the Kubernetes cluster CA certificate from the current kubeconfig:

```
kubectl config view --raw --minify \
-o jsonpath='{.clusters[0].cluster.certificate-authority-data}' \
| base64 -d > ca.crt
```

Verification

Verify that the certificate file was created successfully:

```
cat ca.crt
```

Expected Output

The file should contain a valid PEM formatted certificate similar to:

```
-----BEGIN CERTIFICATE-----
```

```
MIIC...
```

```
...
```

```
-----END CERTIFICATE-----
```

Explanation

The extracted CA certificate will be provided to Vault during Kubernetes Authentication configuration using the `kubernetes_ca_cert` parameter.

This allows Vault to establish a trusted TLS connection to the Kubernetes API server.

Why This Step Is Important

If the CA certificate is missing or incorrect:

- Vault may fail to communicate with the Kubernetes API server.
- TokenReview requests may fail.
- Kubernetes Authentication may return TLS or authentication-related errors.

Always verify that the CA certificate used by Vault matches the CA certificate used by the Kubernetes cluster.

Step 5: Identify and Validate the Kubernetes API Endpoint

Purpose

Vault communicates with the Kubernetes API server using the address specified in the `kubernetes_host` configuration parameter.

Before configuring Vault, verify that:

- The Kubernetes API server is reachable.
- Network connectivity exists.
- The API server certificate is valid.
- The API server certificate contains the public IP address or hostname that Vault will use.

Identify the Kubernetes API Endpoint

Retrieve the public IP address of the Kubernetes node:

```
curl ifconfig.me
```

Example output:

```
<public-ip>
```

Verify Kubernetes API Connectivity

Test connectivity to the Kubernetes API server:

```
curl -k https://<public-ip>:6443/version
```

Expected response:

```
{
  "kind": "Status",
  "status": "Failure",
  "message": "Unauthorized",
  "reason": "Unauthorized",
  "code": 401
}
```

Why HTTP 401 Is Expected

The Kubernetes API server requires authentication.

Receiving an HTTP 401 Unauthorized response confirms that:

- The API server is reachable.
- The API server is listening on port 6443.
- Network connectivity is working correctly.

Validate the Kubernetes API Server Certificate

Inspect the Kubernetes API server certificate:

```
openssl x509 \
-in /var/lib/rancher/k3s/server/tls/serving-kube-apiserver.crt \
-text -noout | grep -A2 "Subject Alternative Name"
```

Example output:

X509v3 Subject Alternative Name:

DNS:kubernetes

DNS:kubernetes.default

DNS:kubernetes.default.svc

DNS:kubernetes.default.svc.cluster.local

IP Address:127.0.0.1

IP Address:<internal-node-ip>

IP Address:<cluster-service-ip>

Important Validation

Verify whether the public IP address that Vault will use is present in the Subject Alternative Name (SAN) list.

Example:

Vault Kubernetes Host:

https://<public-ip>:6443

If the public IP is missing from the SAN list, Vault may experience certificate validation issues when communicating with the Kubernetes API server.

Resolution for Missing SAN Entry

Create or update the K3s configuration:

```
cat > /etc/rancher/k3s/config.yaml <<EOF
```

```
tls-san:
```

```
- "<public-ip>"
```

```
EOF
```

Restart K3s:

```
systemctl restart k3s
```

Verification After Restart

Verify that the certificate has been regenerated:

```
openssl x509 \
```

```
-in /var/lib/rancher/k3s/server/tls/serving-kube-apiserver.crt \
```

```
-text -noout | grep -A2 "Subject Alternative Name"
```

Expected output:

X509v3 Subject Alternative Name:

...

IP Address:<public-ip>

...

Explanation

The Kubernetes API server certificate must contain the address used by Vault in the `kubernetes_host` parameter.

If the public IP address or hostname is not included in the certificate SAN list, TLS validation may fail during Kubernetes authentication operations.

Root Cause Example

In this lab environment, authentication failures were traced to a Kubernetes API server certificate SAN configuration issue. The public IP address used by Vault was not present in the API server certificate SAN list. Updating the K3s `tls-san` configuration and regenerating the certificate resolved the issue.

Step 6: Configure Kubernetes Authentication in Vault

Purpose

Vault must be configured to communicate with the Kubernetes API server in order to validate Service Account JWT tokens during authentication.

This configuration establishes trust between Vault and the Kubernetes cluster by providing:

- A Token Reviewer JWT
- Kubernetes API endpoint
- Kubernetes CA certificate
- Issuer validation settings

Enable Kubernetes Authentication Method

If the Kubernetes Authentication method is not already enabled, enable it using the following command:

```
vault auth enable -path=kb-lab kubernetes
```

Configure Kubernetes Authentication

Configure Vault with the Kubernetes cluster details:

```
vault write auth/kb-lab/config \  
token_reviewer_jwt="<JWT_FROM_STEP_3>" \  
kubernetes_host="https://<public-ip>:6443" \  
kubernetes_ca_cert=@ca.crt \  
disable_iss_validation=true
```

Parameter Explanation

token_reviewer_jwt: Service Account token used by Vault to perform TokenReview requests

kubernetes_host: Kubernetes API server endpoint

kubernetes_ca_cert: Kubernetes cluster CA certificate

disable_iss_validation: Disables issuer validation when required by specific Kubernetes distributions

Validation

Verify that the configuration was applied successfully:

```
vault read auth/kb-lab/config
```

Expected Output

Verify the following values:

```
token_reviewer_jwt_set = true
```

```
kubernetes_host = https://<public-ip>:6443
```

Explanation

When a workload attempts to authenticate using Kubernetes Authentication, Vault uses the configured Token Reviewer JWT to communicate with the Kubernetes API server and perform a TokenReview operation.

The Kubernetes API server validates the Service Account token and returns the authentication result to Vault.

If any of the following are incorrect:

- Token Reviewer JWT
- Kubernetes API endpoint
- CA certificate
- Network connectivity
- TLS certificate validation

authentication requests may fail.

Troubleshooting

If authentication fails:

1. Verify Kubernetes API connectivity.
2. Verify the CA certificate.
3. Verify TokenReview permissions.
4. Verify the Service Account configuration.

5. Verify that the Kubernetes API certificate contains the public IP or hostname used in `kubernetes_host`.
6. Verify that the reviewer token has not expired.

Note:

A successful execution of:

```
#vault write auth/kb-lab/config ...
```

only confirms that the configuration was accepted by Vault.

It does not validate end-to-end authentication.

Authentication must be verified separately using a Service Account JWT and a login request.

Step 7: Create a Vault Policy

Purpose

Vault policies define what authenticated users are allowed to access.

After Kubernetes authentication succeeds, Vault assigns policies to the generated Vault token. These policies determine which secrets and paths can be accessed.

Create Policy File

Create a policy file named `vault-policy.hcl`:

```
path "secret/data/*" {  
  capabilities = ["read"]  
}
```

Upload Policy to Vault

```
vault policy write vault-policy vault-policy.hcl
```

Validation

Verify that the policy exists:

```
vault policy read vault-policy
```

Expected Result

The policy should display the configured path and capabilities.

Explanation

When a Kubernetes workload successfully authenticates to Vault, Vault issues a token containing one or more policies.

These policies control the permissions granted to the authenticated workload.

Step 8: Create a Kubernetes Authentication Role

Purpose

A Vault Kubernetes Authentication Role maps Kubernetes Service Accounts to Vault policies.

When a Service Account JWT is presented to Vault, Vault verifies:

- Service Account name
- Namespace
- Audience (if configured)

If validation succeeds, Vault issues a Vault token with the configured policies.

Create Kubernetes Role

```
vault write auth/kb-lab/role/vault-k8s-role \
bound_service_account_names=<service-account-name> \
bound_service_account_namespaces=<namespace-name> \
audience="k3s" \
policies=vault-policy \
ttl=1h
```

Parameter Explanation

Parameter	Description
bound_service_account_names	Allowed Kubernetes Service Account

Parameter	Description
bound_service_account_namespaces	Namespace containing the Service Account
audience	Expected JWT audience
policies	Vault policies assigned after successful login
ttl	Lifetime of the generated Vault token

Validation

```
vault read auth/kb-lab/role/vault-k8s-role
```

Expected Output

```
bound_service_account_names = [<service-account-name>]
```

```
bound_service_account_namespaces = [<namespace-name>]
```

```
audience = k3s
```

```
policies = [vault-policy]
```

```
ttl = 1h
```

Explanation

This role establishes the trust relationship between Kubernetes and Vault.

Only JWTs generated by the configured Service Account and Namespace will be accepted. Upon successful authentication, Vault issues a token containing the configured policies.

Step 9: Authenticate Using a Kubernetes Service Account JWT

Purpose

This step validates the complete Kubernetes Authentication workflow between Vault and Kubernetes.

A successful login confirms that:

- Vault can communicate with the Kubernetes API server.
- TokenReview permissions are configured correctly.
- Service Account validation is working.

- Vault role mappings are correct.
- Policies are being assigned successfully.

Generate a Fresh JWT

Generate a Service Account token:

```
kubectl create token <service-account-name> -n <namespace-name>
```

Copy the generated JWT for use in the authentication request.

Authenticate with Vault

```

vault write auth/kb-lab/login \
role=vault-k8s-role \
jwt="<FRESH_JWT>"

```

Expected Output

Successful authentication returns output similar to:

Key	Value
---	-----
token	hvs.xxxxxxxxxxxxxx
token_accessor	xxxxxxxxxxxxxx
token_duration	1h
token_policies	["default" "vault-policy"]
identity_policies	[]
renewable	true

Verification

Authentication is considered successful when Vault returns:

- token
- token_accessor
- token_duration
- token_policies

and does not return:

permission denied

What Successful Authentication Confirms

A successful login confirms that:

- ✓ Vault can reach the Kubernetes API server.
- ✓ Kubernetes TokenReview requests are functioning correctly.
- ✓ Service Account validation is successful.
- ✓ Namespace and Service Account mappings are correct.
- ✓ Vault role configuration is correct.
- ✓ Vault policy assignment is working.
- ✓ Kubernetes Authentication is functioning as expected.

Troubleshooting

If authentication fails:

1. Verify Service Account existence.
2. Verify ClusterRoleBinding configuration.
3. Verify TokenReview permissions.
4. Verify Kubernetes API connectivity.
5. Verify Kubernetes CA certificate.
6. Verify Vault Kubernetes Auth configuration.
7. Verify Vault Role configuration.
8. Verify JWT audience settings.
9. Verify Kubernetes API certificate SAN entries.

Common error:

Code: 403

permission denied

When troubleshooting 403 errors, verify:

- TokenReview permissions
- Service Account mapping
- Namespace mapping
- JWT audience configuration
- Kubernetes API certificate SAN validation
- Reviewer JWT validity

Conclusion

This document demonstrated the complete process of configuring Kubernetes Authentication between HCP Vault and a K3s Kubernetes cluster.

The guide covered:

- Service Account creation
- TokenReview permission configuration
- Reviewer JWT generation
- Kubernetes CA certificate extraction
- Kubernetes API endpoint validation
- API certificate SAN verification
- Vault Kubernetes Authentication configuration
- Vault Policy creation
- Vault Role creation
- End-to-end authentication testing

Following these steps allows workloads running in Kubernetes to authenticate securely with Vault using Service Account tokens.

Lessons Learned

1. Successful Vault configuration does not guarantee successful authentication.
2. TokenReview permissions are required for Vault to validate Kubernetes JWTs.
3. The Kubernetes CA certificate must be trusted by Vault.

4. Kubernetes API connectivity should always be validated before authentication testing.
5. Certificate SAN validation is an important troubleshooting step when using public IP addresses or custom hostnames.
6. OpenSSL can be used to inspect and validate Kubernetes API server certificates.
7. End-to-end authentication testing should always be performed after configuration changes.

Author: Chirag Jain

Published: June 2026

Version: 1.0

Document Type: Technical Knowledge Base (KB)

Environment: Personal Lab Environment

Purpose: Learning, validation, troubleshooting, and knowledge sharing.

Version	Date	Author	Description
1.0	June 2026	Chirag Jain	Initial publication

Acknowledgements

This guide was developed through hands-on testing and validation in a personal lab environment to better understand Vault Kubernetes Authentication behavior, troubleshooting techniques, and operational best practices.

Copyright © 2026 Chirag Jain

This article is intended for educational and knowledge-sharing purposes. If reproducing or republishing substantial portions of this document, please provide appropriate attribution to the original author.